

CrossTwin: Function and Equation Registry

A traceability record between the DPSIR causal graph and its computational implementation

CrossTwin Project

September 23, 2026

Abstract

This document catalogs every quantitative computation currently implemented in the CrossTwin platform’s domain apps (`administrative`, `watersupply`, `housing`, `urban_heat`), records the exact equation each function evaluates, the variables and their units, the source data model(s), and the corresponding edge(s) in the project’s Driver–Pressure–State–Impact–Response (DPSIR) causal graph (`core/DAG.dot`). It is intended as a working reference while preparing a manuscript describing CrossTwin’s indicator methodology, and as a running record of which causal relationships are backed by executable derivation logic versus flat, manually entered fields. Status tags (`implemented` / `partial` / `gap`) follow the coverage audit in `TODO.md`.

Contents

1	Scope and conventions	2
2	Administrative app — population projection	2
2.1	Compound annual growth rate — <code>extrapolate()</code>	3
2.2	City-level growth factor — <code>_city_growth_factor()</code>	3
2.3	District/Neighbourhood projection — <code>population_by_year()</code>	3
2.4	Province projection	4
2.5	What-if growth adjustment — <code>_adjust()</code>	4
2.6	City annual growth rate — <code>annual_growth_rate()</code>	4
2.7	Population and density cascade — <code>City.save()</code> , <code>Province.save()</code>	4
2.8	Urban area cascade and source computation — <code>administrative/signals.py</code> , <code>physicalEnv/signals.py</code>	
3	Water supply app	5
3.1	Per-capita consumption — <code>_get_consumption_capita()</code>	5
3.2	Total water demand — <code>calculate_total_demand()</code> , <code>TotalWaterDemand.save()</code>	6
3.3	Total consumption — <code>ConsumptionCapita.save()</code>	6
3.4	Total extraction — <code>calculate_total_extraction()</code>	6
3.5	Total production — <code>calculate_total_production_day()</code>	6
3.6	Supply security & service time	7
3.7	Energy consumption — <code>calculate_energy_consumption()</code>	7
3.8	Pump emissions — <code>ExtractionWater.save()</code>	7
3.9	Extraction OPEX and environmental cost — <code>ExtractionWater.save()</code>	7
3.10	Consolidated OPEX — <code>OPEX.save()</code>	8
3.11	OPEX recovery (dashboard) — <code>calculate_opec_recovery()</code>	8

3.12	Billed consumption and revenue — <code>MeteredResidential.save()</code>	8
3.13	Collection ratio — <code>calculate_collection_ratio()</code>	8
3.14	Coverage — <code>calculate_coverage()</code> , <code>CoverageWaterSupply.save()</code>	9
3.15	Non-Revenue Water and ILI	9
3.16	Water quality — <code>calculate_water_quality()</code>	9
3.17	Available fresh water and drought area	10
3.18	Pipe geometry — <code>PipeNetwork.save()</code>	10
4	Housing app	10
4.1	Supply/demand deficit	10
4.2	Pipeline units — <code>calculate_new_units()</code>	10
4.3	Mortgage indicators — <code>calculate_mortgage_indicators()</code>	11
4.4	House Price Index — <code>calculate_house_price_index()</code>	11
4.5	Affordability index — <code>HousingAffordability.save()</code>	11
4.6	Zoning composition — <code>calculate_zoning()</code>	11
4.7	Property market and vacancy — <code>calculate_property_indicators()</code>	12
5	Urban heat app	12
5.1	Green area and vegetation coverage	12
5.2	Urban morphology / canyon aspect proxy — <code>calculate_urban_morphology()</code> . . .	12
5.3	Raster statistics — <code>_raster_stats_sql()</code> , <code>get_thermal_indices()</code>	12
5.4	Thermal stress classification	13
5.5	Nature-Based Solutions coverage — <code>calculate_nbs_coverage()</code>	13
6	Summary table	13

1 Scope and conventions

Each entry below documents one function or model `save()` method from the codebase. Entries are grouped by Django app, matching the project’s domain decomposition. For every entry we record:

- **Location** — file and line number in the repository at time of writing.
- **Equation** — the exact arithmetic the code performs, in mathematical notation.
- **Variables** — symbol, meaning, unit, and source field/model.
- **DAG edge(s)** — the causal edge(s) in `core/DAG.dot` this computation is claimed to implement.
- **Status** — whether the DAG edge is genuinely derived from its source field (**implemented**), derived but with a known caveat or partial dependency (**partial**), or claimed in a docstring without being computed (**gap**).

Units follow the project convention: population counts are dimensionless integers; areas are km² unless noted; water volumes are m³ or Mm³ (explicitly distinguished per entry, since several models mix the two); monetary values are EUR.

2 Administrative app — population projection

Source: `administrative/population.py`, `administrative/models.py`. Population data originates from CBS table 85173NED (*Regionale prognose 2023–2050*), published per *gemeente* (city) [1].

2.1 Compound annual growth rate — `extrapolate()`

Location: `administrative/population.py:67`

Projects a city's population for a year outside the imported CBS horizon, using the compound annual growth rate implied by the earliest and latest imported years.

$$r = \left(\frac{v_1}{v_0} \right)^{\frac{1}{y_1 - y_0}} - 1 \quad (1)$$

$$P(y) = v_a (1 + r)^{(y - y_a)} \quad (2)$$

Variables:

- y_0, y_1 — earliest / latest year with an imported `PopulationProjection` record for the city
- v_0, v_1 — projected population at y_0, y_1
- r — implied compound annual growth rate
- y_a, v_a — anchor year/value: (y_1, v_1) if $y > y_1$, else (y_0, v_0)

DAG edge: *DAG edge: Population_Growth → Total_Population* — **implemented**

2.2 City-level growth factor — `_city_growth_factor()`

Location: `administrative/population.py:103`

Ratio of a city's CBS-projected population between a baseline year and a target year, used to scale District/Neighbourhood populations without relying on possibly-incomplete local administrative coverage.

$$f(y) = \frac{P_{\text{city}}(y)}{P_{\text{city}}(y_{\text{baseline}})}, \quad y_{\text{baseline}} = \begin{cases} 2025 & \text{if imported} \\ \min(y_{\text{imported}}) & \text{otherwise} \end{cases} \quad (3)$$

DAG edge: *DAG edge: Population_Growth → Total_Population* (District/Neighbourhood level) — **implemented**

2.3 District/Neighbourhood projection — `population_by_year()`

Location: `administrative/population.py:124`

$$P_{\text{unit}}(y) = P_{\text{unit}}^{\text{current}} \times f(y) \quad (4)$$

Applied to the unit's own accurate current population rather than its share of `city.currentPopulation`, because the latter reflects only the districts/neighbourhoods imported locally, not CBS's whole-city total (see docstring, `administrative/population.py:11--22`).

Fallback (fewer than two imported years for the city): share-of-city approach,

$$P_{\text{unit}}(y) = \frac{P_{\text{unit}}^{\text{current}}}{P_{\text{city}}^{\text{current}}} \times P_{\text{city}}(y) \quad (5)$$

DAG edge: *DAG edge: Population_Growth → Total_Population* — **implemented**

2.4 Province projection

Location: administrative/population.py:132--138

$$P_{\text{province}}(y) = \sum_{c \in \text{cities of province}} P_c(y) \quad (6)$$

DAG edge: *DAG edge: Population_Growth → Total_Population* — implemented

2.5 What-if growth adjustment — `_adjust()`

Location: administrative/population.py:59

$$v_{\text{adjusted}} = v \times \left(1 + \frac{g}{100}\right)^{\max(y-2025, 0)} \quad (7)$$

where g is `growth_adjust_pct`, a user-supplied what-if slider bounded to $[-5, +5]$ (`population_params()`, administrative/population.py:205).

2.6 City annual growth rate — `annual_growth_rate()`

Location: administrative/population.py:90 (as of the `City.popGrowthRate` implementation)

$$\text{popGrowthRate} = \left(\left(\frac{v_1}{v_0} \right)^{\frac{1}{y_1 - y_0}} - 1 \right) \times 100 \quad (8)$$

Same CAGR as `extrapolate()`, expressed in percent per year and written onto `City.popGrowthRate` in `City.save()` (administrative/models.py) whenever at least two `PopulationProjection` years exist for the city.

DAG edge: *DAG edge: Population_Growth → Total_Population* (`City.popGrowthRate`) — implemented

2.7 Population and density cascade — `City.save()`, `Province.save()`

Location: administrative/models.py

$$P_{\text{city}} = \sum_{d \in \text{districts of city}} P_d \quad (\text{only if districts report a nonzero total}) \quad (9)$$

$$P_{\text{province}} = \sum_{c \in \text{cities of province}} P_c \quad (10)$$

$$\rho = \frac{P}{A}, \quad A = \frac{\text{geom.area}}{10^6} \text{ km}^2 \quad (11)$$

Density ρ in people/km². The Neighbourhood → District → City → Province cascade is signal-driven (administrative/signals.py), using `QuerySet.update()` rather than `.save()` to avoid re-triggering `post_save` recursively.

DAG edge: (population aggregation, not a numbered DAG node) — implemented

2.8 Urban area cascade and source computation — administrative/signals.py, physicalEnv/signals.py

Location: administrative/signals.py (cascade), physicalEnv/signals.py (source)

$$A_{\text{nbhd}}^{\text{urban}} = \sum_{v \in \text{qualifying LandCoverVector, latest year}} |\text{geom}_{\text{nbhd}} \cap \text{geom}_v| / 10^6 \text{ km}^2 \quad (12)$$

$$A_{\text{district}}^{\text{urban}} = \sum_{n \in \text{neighbourhoods of district}} A_n^{\text{urban}} \quad (13)$$

$$A_{\text{city}}^{\text{urban}} = \sum_{d \in \text{districts of city}} A_d^{\text{urban}} \quad (14)$$

$$A_{\text{province}}^{\text{urban}} = \sum_{c \in \text{cities of province}} A_c^{\text{urban}} \quad (15)$$

A `physicalEnv.LandCoverVector` polygon qualifies if its `land_cover_type.class_name` case-insensitively contains any of *urban fabric*, *road*, *rail*, *transport*, restricted to the most recent **year** present for that city (older survey years are not summed, to avoid double-counting repeat surveys). The intersection area is computed in PostGIS via `ST_Intersection` (Django’s `Intersection/Area` functions). Neighbourhood → District → City → Province propagation reuses the same `_recompute_population()` helper as the population cascade above, generalised to sum `urban_area` alongside `currentPopulation` in one aggregate; both use `QuerySet.update()`, not `.save()`. Unlike population, an aggregate with no children reporting a value is left `None` (unknown) rather than coerced to 0. Because a `LandCoverVector` write only fires `post_save` on itself (not on `Neighborhood`), the `physicalEnv` signal re-enters the `administrative` cascade manually after writing each affected `Neighborhood`’s `urban_area`.

DAG edge: `LandCover` → `Urbanization` is the closest existing edge in `core/DAG.dot`, but it runs in the opposite direction of what this computes (`LandCover` only feeds a not-yet-modelled `Urbanization` node, per `TOD0.md`), and no `Neighborhood/District/City/Province/LandCoverVector` nodes exist in the DAG at all — **partial** (cascade and source computation fully implemented; no `Urbanization` model backs a DAG edge for it)

3 Water supply app

Source: `watersupply/calculations.py`, `watersupply/models.py`, `watersupply/signals.py`.

3.1 Per-capita consumption — `_get_consumption_capita()`

Location: `watersupply/calculations.py:16`

$$\bar{c} = \text{avg}_{c \in \text{cities}(u)} (\text{consumption_capita_L_d}), \quad [\bar{c}] = \text{L/person/day} \quad (16)$$

3.2 Total water demand — `calculate_total_demand()`, `TotalWaterDemand.save()`

Location: `watersupply/calculations.py:27`; `watersupply/models.py:70`

$$D_{\text{day}} = \frac{\bar{c}}{1000} \times P \quad [\text{m}^3/\text{day}] \quad (17)$$

$$\text{demandDay} = \frac{P \times \bar{c}}{10^9} \quad [\text{Mm}^3/\text{day}] \quad (18)$$

$$\text{demandYR} = \text{demandDay} \times 365 \quad (19)$$

where P is `city.currentPopulation`. Note the two representations differ by a factor of 10^6 (m^3 vs. Mm^3); `calculate_supply_security()` converts explicitly (§3.6).

DAG edges: *DAG edge: `Total_Population` → `Total_Water_Demand`, DAG edge: `Consumption-Capita` → `Total_Water_Demand` — implemented*

3.3 Total consumption — `ConsumptionCapita.save()`

Location: `watersupply/models.py:38`

$$\text{total_consumption_m3_yr} = \frac{\text{consumption_capita_L_d}}{1000} \times P \times 365 \quad (20)$$

DAG edges: *DAG edge: `Total_Population` → `Total_Consumption`, DAG edge: `ConsumptionCapita` → `Total_Consumption` — implemented*

3.4 Total extraction — `calculate_total_extraction()`

Location: `watersupply/calculations.py:38`

$$E = \left(\sum_{w \in \text{active wells} \cap \text{geom}(u)} q_w \right) \times 86400, \quad [q_w] = \text{m}^3/\text{s}, [E] = \text{m}^3/\text{day} \quad (21)$$

DAG edge: *DAG edge: `Available_FW` → `Total_Extraction` — implemented*

3.5 Total production — `calculate_total_production_day()`

Location: `watersupply/calculations.py:51`

$$\Pi = E + I, \quad I = \sum_{\text{active imports}} \text{quantity_m3_d} \quad (22)$$

DAG edges: *DAG edge: `Total_Extraction` → `Total_Water_Prod`, DAG edge: `Imported_Water` → `Total_Water_Prod` — implemented*

3.6 Supply security & service time

Location: `calculate_supply_security()` (`watersupply/calculations.py:73`); `SupplySecurity.save()` (`watersupply/models.py:107`)

$$D_{\text{m}^3/\text{day}} = \text{demandDay} \times 10^6 \quad (23)$$

$$S = \frac{\Pi}{D_{\text{m}^3/\text{day}}} \quad (\text{ratio; } \times 100 \text{ for } \%) \quad (24)$$

$$T_{\text{service}} = \begin{cases} 24 & \Pi \geq D_{\text{m}^3/\text{day}} \\ 24 \times \frac{\Pi}{D_{\text{m}^3/\text{day}}} & \Pi < D_{\text{m}^3/\text{day}} \end{cases} \quad [\text{h/day}] \quad (25)$$

DAG edges: *DAG edge: Total_Water_Demand → Supply_Security*, *DAG edge: Total_Water_Prod → Supply_Security*, *DAG edge: Supply_Security → Service_Time* — **implemented**

3.7 Energy consumption — `calculate_energy_consumption()`

Location: `watersupply/calculations.py:100`

$$K = \sum_w (\text{pumpEnergyRate_kWh_h}_w \times \text{OperationTime_h_day}_w) + 1000 \times \sum \text{EnergyConsumption_MW_day} \quad (26)$$

[K] = kWh/day. **DAG edges:** *DAG edge: Total_Extraction → Energy_Consumption*, *DAG edge: Total_Water_Prod → Energy_Consumption*, *DAG edge: Energy_Cost → Energy_Consumption* — **partial** (Energy_Cost side of the edge is not read back into this function)

3.8 Pump emissions — `ExtractionWater.save()`

Location: `watersupply/models.py:317`

$$R_{\text{CO}_2} = \text{pumpEnergyRate_kWh_h} \times \text{pumpEmissionFactor_kg_CO}_2_\text{kWh} \quad (27)$$

$$M_{\text{day}} = R_{\text{CO}_2} \times \text{OperationTime_h_day}, \quad M_{\text{yr}} = M_{\text{day}} \times 365 \quad (28)$$

DAG edge: *DAG edge: Total_Extraction → CO2_Emission* — **implemented** (per-well; `calculate_co2_emission()` sums across wells)

3.9 Extraction OPEX and environmental cost — `ExtractionWater.save()`

Location: `watersupply/models.py:346--357`

$$\text{opex_EUR_m3} = \text{labor} + \text{energy} + \text{chemicals} + \text{tax} \quad (\text{only when at least one component is known}) \quad (29)$$

$$\text{CO}_2 \text{ cost} = \frac{M_{\text{day}}}{V_{\text{day}}} \times \text{price}_{\text{CO}_2}, \quad V_{\text{day}} = q \times \text{OperationTime_h_day} \times 3600 \quad (30)$$

3.10 Consolidated OPEX — `OPEX.save()`

Location: `watersupply/models.py:640`

$$\text{Cost}_{\text{extraction}} = \sum (\text{opex_EUR_m3} \times \text{current_extraction_Mm3_yr} \times 10^6) \quad (31)$$

$$\text{Cost}_{\text{imported}} = \sum (\text{quantity_m3_d} \times 365 \times \text{price_EUR_m3}) \quad (32)$$

$$\text{Cost}_{\text{treatment}} = \frac{\text{UnitaryOPEX_EUR_m3}}{\text{year}} \times V_{\text{yr}} \quad (33)$$

$$\text{Cost}_{\text{network}} = \sum (\text{maintenanceCost_EUR_km} \times \text{length_km}) \quad (34)$$

$$\text{totalOPEX_EUR} = \text{Cost}_{\text{extraction}} + \text{Cost}_{\text{imported}} + \text{Cost}_{\text{treatment}} + \text{Cost}_{\text{network}} \quad (35)$$

$$\text{UnitaryOPEX_EUR_m3} = \frac{\text{totalOPEX_EUR}}{V_{\text{yr}}}, \quad V_{\text{yr}} = V_{\text{extraction}} + V_{\text{imported}} \quad (36)$$

`TotalWaterProduction`'s own energy cost is deliberately excluded, since it is already embedded in `opex_EUR_m3` (see inline comment, `watersupply/models.py`).

$$\text{OPEX_recovered_PCT} = \frac{\sum \text{Recovery_EUR}}{\text{totalOPEX_EUR}} \times 100 \quad (37)$$

DAG edges: *DAG edge: Total_Extraction/Total_Water_Prod → OPEX*, *DAG edge: Imported_Water → OPEX*, *DAG edge: WT_Cost → OPEX*, *DAG edge: Network → OPEX*, *DAG edge: Metered_Res_Water → OPEX_Recovery* — **implemented**

3.11 OPEX recovery (dashboard) — `calculate_opex_recovery()`

Location: `watersupply/calculations.py:205`

$$\text{Recovery\%} = \frac{\sum_{\text{metered users}} \text{Recovery_EUR}}{\text{totalOPEX_EUR}_{\text{year}}} \times 100 \quad (38)$$

DAG edges: *DAG edge: OPEX → OPEX_Recovery*, *DAG edge: CollectionRatio → OPEX_Recovery*, *DAG edge: NRW → OPEX_Recovery* — **partial** (computes directly from stored totals; does not call `calculate_collection_ratio()` or reference `NRW`, despite the docstring)

3.12 Billed consumption and revenue — `MeteredResidential.save()`

Location: `watersupply/models.py:172`

$$\gamma = \frac{\text{collected_meters}}{\text{installed_meters}} \quad (39)$$

$$V_{\text{yr}} = \frac{\text{consumption_capita_L_d}}{1000} \times 365 \times \text{populationServed} \times \gamma \quad (40)$$

$$\text{Recovery_EUR} = V_{\text{yr}} \times \text{userTariff_EUR_m3} \quad (41)$$

Uses the city's most recent `ConsumptionCapita` record (not filtered by year), applied to local `populationServed` rather than the whole-city `total_consumption_m3_yr`.

3.13 Collection ratio — `calculate_collection_ratio()`

Location: `watersupply/calculations.py:184`

$$\text{CR\%} = \frac{\sum \text{collected_meters}}{\sum \text{installed_meters}} \times 100 \quad (42)$$

DAG edges: *DAG edge: Metered_Res_Water* $\rightarrow$ *CollectionRatio*, *DAG edge: User_Acceptance_WS* $\rightarrow$ *CollectionRatio*, *DAG edge: Water_Tariff_Afford* $\rightarrow$ *CollectionRatio* — **partial** (ignores userAffordability_PCT and acceptance rate despite the docstring)

3.14 Coverage — `calculate_coverage()`, `CoverageWaterSupply.save()`

Location: `watersupply/calculations.py:239`; `watersupply/models.py:448`

$$G_{\text{buffer}} = \bigcup_{p \in \text{pipes}(c)} \text{buffer}(p, 500 \text{ m}) \quad (43)$$

$$A_{\text{covered}} = \frac{\text{area}(G_{\text{buffer}})}{10^6} [\text{km}^2] \quad (44)$$

$$\text{Coverage}\% = \frac{H_{\text{covered}}}{H_{\text{total}}} \times 100 \quad (45)$$

where H_{covered} counts households in `UsersLocation` points spatially within G_{buffer} , and H_{total} sums `usersTotal` across the city's neighbourhoods. **DAG edges:** *DAG edge: Network* $\rightarrow$ *Coverage_WS_Area*, *DAG edge: CityArea* $\rightarrow$ *Coverage_WS_Area*, *DAG edge: NumberUsers* $\rightarrow$ *Coverage_WS* — **implemented**

3.15 Non-Revenue Water and ILI

Location: `calculate_nrw()` (`watersupply/calculations.py:275`); `NonRevenueWater.save()` (`watersupply/models.py:551`)

$$\text{NRW}_{\text{apparent}} = \sum_{\text{type}=A} \text{loss_Quantity_m3} \quad (46)$$

$$\text{NRW}_{\text{real}} = \sum_{\text{type}=R} \text{loss_Quantity_m3} \quad (47)$$

$$\text{CARL} = \sum_{\text{type}=R, \text{ year}} \text{loss_Quantity_m3} \quad (48)$$

$$\text{UARL} = \sum_{\text{type}=R, \text{ year}} \text{loss_Quantity_m3} \times \frac{\text{UnavoidableLossses_PCT}}{100} \quad (49)$$

$$\text{ILI} = \frac{\text{CARL}}{\text{UARL}} \quad (50)$$

The Infrastructure Leakage Index formulation follows the IWA Water Loss Task Force methodology [3]. **DAG edges:** *DAG edge: Real_Losses* $\rightarrow$ *NRW*, *ILI*, *DAG edge: Apparent_Losses* $\rightarrow$ *NRW*, *ILI* — **partial** (ILI is only defined for Real losses; *DAG edge: Apparent_Losses* $\rightarrow$ *ILI* is unimplemented; `NonRevenueWater` records are entered directly rather than linked to `PipeNetwork`, so *DAG edge: Network* $\rightarrow$ *Real_Losses* is a **gap**)

3.16 Water quality — `calculate_water_quality()`

Location: `watersupply/calculations.py:144`

$$\text{Compliance}\% = \frac{\sum \text{samplesWaterQuality_OK}}{\sum \text{samplesWaterQualityTaken}} \times 100 \quad (51)$$

DAG edges: *DAG edge:* `WT_Efficiency` $\rightarrow$ `Samples_WQ`, *DAG edge:* `Samples_Taken` $\rightarrow$ `Samples_WQ`, *DAG edge:* `Samples_WQ` $\rightarrow$ `User_Acceptance_WS` — **gap** (`acceptance_rate` in the return value is `avg(acceptanceRate)`, an independent stored field unrelated to the computed compliance percentage)

3.17 Available fresh water and drought area

Location: `watersupply/calculations.py:308, 323`

$$AFW = \sum_{\text{intersecting } u} \text{totalQuantity_Mm3} \quad (52)$$

$$A_{\text{drought}} = \sum_{\text{intersecting } u, \text{ year}} \text{areaAffected_km2} \quad (53)$$

DAG edges: *DAG edge:* `Infiltration` $\rightarrow$ `Available_FW`, *DAG edge:* `Meteorology` $\rightarrow$ `Available_FW`, *DAG edge:* `Total_Extraction` $\rightarrow$ `Area_Drought` — **gap** for the Infiltration side (`AvailableFreshWater.infiltration` is a flat, manually entered field — see `TODO.md`)

3.18 Pipe geometry — `PipeNetwork.save()`

Location: `watersupply/models.py:420`

$$L_{\text{km}} = \frac{\text{length}(\text{geom})}{1000}, \quad d_{\text{in}} = \frac{d_{\text{mm}}}{25.4} \quad (54)$$

4 Housing app

Source: `housing/calculations.py, housing/models.py`.

4.1 Supply/demand deficit

Location: `calculate_supply_demand()` (`housing/calculations.py:14`), `calculate_supply_demand_for_unit()` (`:43`)

$$\text{Deficit} = D - S \quad (55)$$

$$\text{Deficit}\% = \frac{D - S}{D} \times 100 \quad (56)$$

where $S = \text{supply_units}$, $D = \text{demand_units}$ (stored, not derived from population). **DAG edges:** *DAG edge:* `Urbanization` $\rightarrow$ `Housing_Demand`, *DAG edge:* `New_Units` $\rightarrow$ `Housing_Supply`, *DAG edge:* `Housing_Demand` $\rightarrow$ `House_Price_Index` — **gap** (`demand_units` is a stored value, not derived from §2's population projection; no `Urbanization` model exists)

4.2 Pipeline units — `calculate_new_units()`

Location: `housing/calculations.py:66`

$$U_{\text{completed}} = \sum_{\text{year_expected_completion} \leq y} \text{project_units} \quad (57)$$

$$U_{\text{pipeline}} = \sum_{\text{year_expected_completion} > y} \text{project_units} \quad (58)$$

DAG edges: *DAG edge: Zoning_Status → New_Units, DAG edge: Network → New_Units, DAG edge: New_Units → Housing_Supply, DAG edge: New_Units → LandCover* — **partial** (counts completions/pipeline; never writes back to `LandCover` or reads `Zoning_Status/Network` as inputs)

4.3 Mortgage indicators — `calculate_mortgage_indicators()`

Location: `housing/calculations.py:123`

$$\overline{m} = \text{avg}(\text{monthlyPayment}), \quad \overline{t} = \text{avg}(\text{interestRate}), \quad \overline{L} = \text{avg}(\text{totalLoanAmount}) \quad (59)$$

plus province-level policy fields (`interest_rate`, `LTV_limit`, `LTI_limit`, `mortgageRate`) read as-is. **DAG edges:** *DAG edge: Central_Bank → Mortgage, DAG edge: Credit_Supply → Mortgage, DAG edge: Mortgage → Affordability_Stress, DAG edge: Mortgage → Income_Expenses* — **gap** for the two outgoing edges (averages are exposed on the dashboard but never fed back into `HousingAffordability`)

4.4 House Price Index — `calculate_house_price_index()`

Location: `housing/calculations.py:200`

$$\Delta_{Y \text{ to } Y} \% = \frac{\overline{\text{HPI}}_y - \overline{\text{HPI}}_{y-1}}{\overline{\text{HPI}}_{y-1}} \times 100 \quad (60)$$

DAG edges: *DAG edge: Housing_Demand → House_Price_Index, DAG edge: Property → House_Price_Index* — **gap** (`HousePriceIndex` has no `save()`; `index_value` is a flat stored field, and this function only averages it)

4.5 Affordability index — `HousingAffordability.save()`

Location: `housing/models.py:150`

$$\text{AI} = \frac{\text{medianHousePrice}}{\text{medianIncome}} \quad (61)$$

$$\text{Income}_{\text{disposable}} = \text{medianIncome} - \text{medianExpenditure} \quad (62)$$

The house-price-to-income ratio formulation is the standard affordability metric used in comparative housing-affordability studies (e.g. Demographia’s Median Multiple) [4]. **DAG edges:** *DAG edge: Property → Affordability_Stress, DAG edge: Income_Expenses → Affordability_Stress, DAG edge: Mortgage → Affordability_Stress, DAG edge: Water_Tariff_Afford → Income_Expenses, DAG edge: Rent → Income_Expenses, DAG edge: Mortgage → Income_Expenses* — **gap** (computed only from this model’s own stored fields; never reads `Mortgage`, `Rentals`, or `watersupply` tariff data — an open TODO in `housing/models.py` also questions whether disposable income should replace median income in the numerator’s denominator)

4.6 Zoning composition — `calculate_zoning()`

Location: `housing/calculations.py:285`

$$\text{Area}_{\% \text{ type}} = \frac{\sum_{\text{zone_type}} \text{area}}{\sum_{\text{all zones}} \text{area}} \times 100 \quad (63)$$

DAG edges: *DAG edge: Housing_Demand → Zoning_Status, DAG edge: Zoning_Status → Property, DAG edge: Zoning_Status → New_Units* — **gap** (descriptive aggregation only; no model consumes `Zoning_Status` as an input)

4.7 Property market and vacancy — `calculate_property_indicators()`

Location: `housing/calculations.py:236`

$$\bar{p} = \text{avg}(\text{salePrice_EUR}), \quad \bar{p}_{\text{EUR/m}^2} = \text{avg}(\text{unitaryPrice_EUR_per_sqm}), \quad \bar{v} = \text{avg}(\text{vacancyRate}) \quad (64)$$

DAG edges: *DAG edge: Housing_Supply → Property, DAG edge: Zoning_Status → Property, DAG edge: Accessibility → Property, DAG edge: Property → Affordability_Stress, DAG edge: Property → Rent, DAG edge: Property → House_Price_Index, DAG edge: Property → Mortgage — gap* (no Accessibility model exists; none of the outgoing edges are computed — this function only averages stored fields)

5 Urban heat app

Source: `urban_heat/calculations.py`. Thermal-comfort rasters (SVF, T_{mrt} , PET, UTCI) are expected to arrive pre-computed from an external SOLWEIG run [5]; CrossTwin currently only reads and classifies them, it does not derive one raster from another.

5.1 Green area and vegetation coverage

Location: `calculate_green_area()` (`urban_heat/calculations.py:18`), `calculate_vegetation_coverage()` (`:55`)

$$A_{\text{green,lc}} = A_{\text{unit}} \times \frac{\sum \text{percentage}_{\text{green classes}}}{100} \quad (65)$$

$$A_{\text{green,total}} = A_{\text{parks}} + A_{\text{green,lc}} \quad (66)$$

$$\text{VegCov}\% = \frac{A_{\text{green,total}}}{A_{\text{unit}}} \times 100 \quad (67)$$

`percentage` is stored relative to the *Province* total area regardless of the queried unit (documented limitation, see `TODO.md`). **DAG edges:** *DAG edge: LandCover → Green_Area, DAG edge: Green_Area → Vegetation_Coverage — partial; DAG edge: Vegetation_Coverage → DSM — gap*

5.2 Urban morphology / canyon aspect proxy — `calculate_urban_morphology()`

Location: `urban_heat/calculations.py:75`

$$\text{Aspect ratio} \equiv \frac{H}{W} = \frac{\overline{\text{height_m}}_{\text{buildings}}}{\overline{\text{width}}_{\text{streets}}} \quad (68)$$

The height-to-width (H/W) canyon aspect ratio is the standard urban-canyon geometry descriptor used in urban climatology [6]. **DAG edges:** *DAG edge: Buildings → DSM, Canyon_Aspect, DAG edge: Streets → Canyon_Aspect — partial* (H/W is computed as a scalar proxy over the whole admin unit, not written back onto a Canyon Aspect or DSM model)

5.3 Raster statistics — `_raster_stats_sql()`, `get_thermal_indices()`

Location: `urban_heat/calculations.py:112, 145`

$$\{\min, \max, \mu, \sigma\} = \text{ST_SummaryStats}(\text{ST_Clip}(R, \text{geom}(u))) \quad (69)$$

computed via PostGIS raster functions over the most recently dated raster of each type (LST, T_{mrt} , PET, UTCI, WBGT, SVF, SUHII) intersecting the admin unit. **DAG edges:** *DAG edge: Meteorology* $\rightarrow T_{mrt}$, *PET*, *UTCI*, *DAG edge: DSM* $\rightarrow SVF \rightarrow T_{mrt}$, *PET*, *DAG edge: LST* $\rightarrow PET$, *DAG edge: $T_{mrt} \rightarrow PET$, UTCI* — **gap** (statistics *over* existing rasters only; no raster is derived from another)

5.4 Thermal stress classification

Location: `classify_pet()` (:201), `classify_utci()` (:225), `classify_wbgt()` (:267)

PET thresholds (in °C) follow the physiologically-based 9-class scheme of Matzarakis and Mayer [7]:

$$\text{class(PET)} = \begin{cases} \text{Very Cold Stress} & \text{PET} < 4 \\ \text{Cold Stress} & 4 \leq \text{PET} < 8 \\ \text{Slight Cold Stress} & 8 \leq \text{PET} < 13 \\ \text{No Thermal Stress (cool)} & 13 \leq \text{PET} < 18 \\ \text{Comfortable} & 18 \leq \text{PET} < 23 \\ \text{No Thermal Stress (warm)} & 23 \leq \text{PET} < 29 \\ \text{Slight Heat Stress} & 29 \leq \text{PET} < 35 \\ \text{Moderate Heat Stress} & 35 \leq \text{PET} < 41 \\ \text{Extreme Heat Stress} & \text{PET} \geq 41 \end{cases} \quad (70)$$

UTCI thresholds follow the 10-class scheme defined by the COST Action 730 UTCI framework [8]:

$$\text{class(UTCI)} = \begin{cases} \text{Extreme Cold Stress} & \text{UTCI} < -40 \\ \vdots & \vdots \\ \text{Extreme Heat Stress} & \text{UTCI} \geq 46 \end{cases} \quad (9 \text{ intermediate bands; see } \text{urban_heat/calculations.py}) \quad (71)$$

WBGT bands (°C: 25/28/30/33 thresholds) are marked **provisional** in the source (`urban_heat/calculations.py`) pending confirmation against a named standard.

5.5 Nature-Based Solutions coverage — `calculate_nbs_coverage()`

Location: `urban_heat/calculations.py`:283

$$A_{\text{NBS}} = \sum_{\text{NBS polygons} \cap \text{geom}(u)} \text{area} \quad (72)$$

DAG edges: *DAG edge: UTCI* $\rightarrow NBS$, *DAG edge: PET* $\rightarrow NBS$ — **gap** (counts NBS geometries only; never reads PET or UTCI values to relate NBS placement to thermal stress)

6 Summary table

App	Function	DAG edge(s) (abbrev.)	Status
administrative	<code>extrapolate()</code>	Population.Growth $\rightarrow$ TotalPop	Reported

App	Function	DAG edge(s) (abbrev.)	Status
administrative	<code>_city_growth_factor()</code>	Population_Growth→Total_Pop	implemented
administrative	<code>annual_growth_rate()</code>	Population_Growth→Total_Pop	implemented
administrative	<code>City.save()</code> / <code>Province.save()</code>	(population cascade)	implemented
watersupply	<code>calculate_total_demand()</code> / <code>TotalWaterDemand.save()</code>	Total_Pop→Total_Water_Demanded	implemented
watersupply	<code>ConsumptionCapita.save()</code>	Total_Pop→Total_Consumption	implemented
watersupply	<code>calculate_total_extraction()</code>	Available_FW→Total_Extraction	implemented
watersupply	<code>calculate_total_production()</code>	Total_Extraction/Imported_Water→Total_Water_Prod	implemented
watersupply	<code>calculate_supply_security()</code> / <code>SupplySecurity.save()</code>	Total_Water_Demand/People→Supply_Security→Service_T	implemented
watersupply	<code>calculate_energy_consumption()</code>	Total_Extraction/Prod→Energy_Consumption	implemented
watersupply	<code>ExtractionWater.save()</code>	Total_Extraction→CO2_Emissions (emissions)	implemented
watersupply	<code>OPEX.save()</code>	Extraction/Imported/Water_Cost/Network→OPEX	implemented
watersupply	<code>calculate_opex_recovery()</code>	OPEX/CollectionRatio/NRW→OPEX_Recovery	implemented
watersupply	<code>MeteredResidential.save()</code>	Metered_Res_Water→OPEX_Recovery	implemented
watersupply	<code>calculate_collection_ratio()</code>	Metered_Res_Water/User_Acceptance/Water_Tariff→Col	implemented
watersupply	<code>calculate_coverage()</code> / <code>CoverageWaterSupply.save()</code>	Network/CityArea→CoverageWS_Area	implemented
watersupply	<code>calculate_nrw()</code> / <code>NonRevenueWater.save()</code>	Real/Apparent_Losses→NRW_ILI	implemented
watersupply	<code>calculate_water_quality()</code>	Samples_WQ→User_Acceptance_WS	implemented
watersupply	<code>calculate_available_freshwater()</code>	Water_Use/Population/Meteorology→Available_FW	implemented
housing	<code>calculate_supply_demand()</code>	Urbanization→Housing_Demand	implemented
housing	<code>calculate_new_units()</code>	New_Units→Housing_Supply/LandCover	implemented
housing	<code>calculate_mortgage_indicators()</code>	Central Bank/Credit_Supply→Mortgage	implemented
housing	<code>calculate_house_price_index()</code>	Housing_Demand/Property_gdp	implemented
housing	<code>HousingAffordability.save()</code>	Rent/Mortgage/Water_Tariff/Income_Expenses→Afford	implemented
housing	<code>calculate_zoning()</code>	Zoning_Status→Property/New_Units	implemented
housing	<code>calculate_property_indicators()</code>	Housing_Supply/Accessibility→Property	implemented
urban_heat	<code>calculate_green_area()</code> / <code>calculate_vegetation_coverage()</code>	LandCover→Green_Area→Vegetation_Coverage	implemented
urban_heat	<code>calculate_urban_morphology()</code>	Buildings/Streets→Canyon_Aspect	implemented
urban_heat	<code>get_thermal_indices()</code>	DSM→SVF→Tmrt/PET→UTCI	implemented
urban_heat	<code>calculate_nbs_coverage()</code>	UTCI/PET→NBS	implemented

Notes on use

This document is generated from a manual audit of the codebase at the commit referenced in the accompanying `TODO.md`. Line numbers will drift as the code evolves; re-verify against `grep -n "def save"` in the relevant module before citing a location in a manuscript. The status tags mirror the

coverage figures reported in the project’s DPSIR gap analysis: of 102 edges in `core/DAG.dot`, 28 are backed by real derivation logic as of this audit.

Beyond the CBS 85173NED forecasts used for the `Population_Growth→Total_Population` projection, the external data catalog also offers a finer-grained, Google Earth Engine-hosted population source, the High Resolution Settlement Layer (HRSL) [2], for direct raster ingestion where a sub-neighbourhood estimate is preferable to the CBS city-level forecast.

References

- [1] Centraal Bureau voor de Statistiek (CBS). *Regionale prognose 2023–2050* (Table 85173NED). <https://opendata.cbs.nl/>.
- [2] Facebook Connectivity Lab and Center for International Earth Science Information Network (CIESIN), Columbia University (2016). *High Resolution Settlement Layer (HRSL)*. Source imagery for HRSL Copyright 2016 DigitalGlobe. <https://gee-community-catalog.org/projects/hrsl/>.
- [3] Lambert, A., Brown, T. G., Takizawa, M., & Weimer, D. (1999). A review of performance indicators for real losses from water supply systems. *Journal of Water Supply: Research and Technology-AQUA*, 48(6), 227–237. (International Water Association Water Loss Task Force ILI methodology.)
- [4] Demographia. *International Housing Affordability Survey* — Median Multiple methodology (median house price / median household income). <http://www.demographia.com/>.
- [5] Lindberg, F., Holmer, B., & Thorsson, S. (2008). SOLWEIG 1.0 – Modelling spatial variations of 3D radiant fluxes and mean radiant temperature in complex urban settings. *International Journal of Biometeorology*, 52(7), 697–713.
- [6] Oke, T. R. (1987). *Boundary Layer Climates* (2nd ed.). Routledge. (Urban canyon height-to-width aspect ratio.)
- [7] Matzarakis, A., & Mayer, H. (1996). Another kind of environmental stress: thermal stress. *WHO Newsletter*, 18, 7–10. (Physiological Equivalent Temperature classification.)
- [8] Bröde, P., Fiala, D., Błażejczyk, K., et al. (2012). Deriving the operational procedure for the Universal Thermal Climate Index (UTCI). *International Journal of Biometeorology*, 56(3), 481–494.